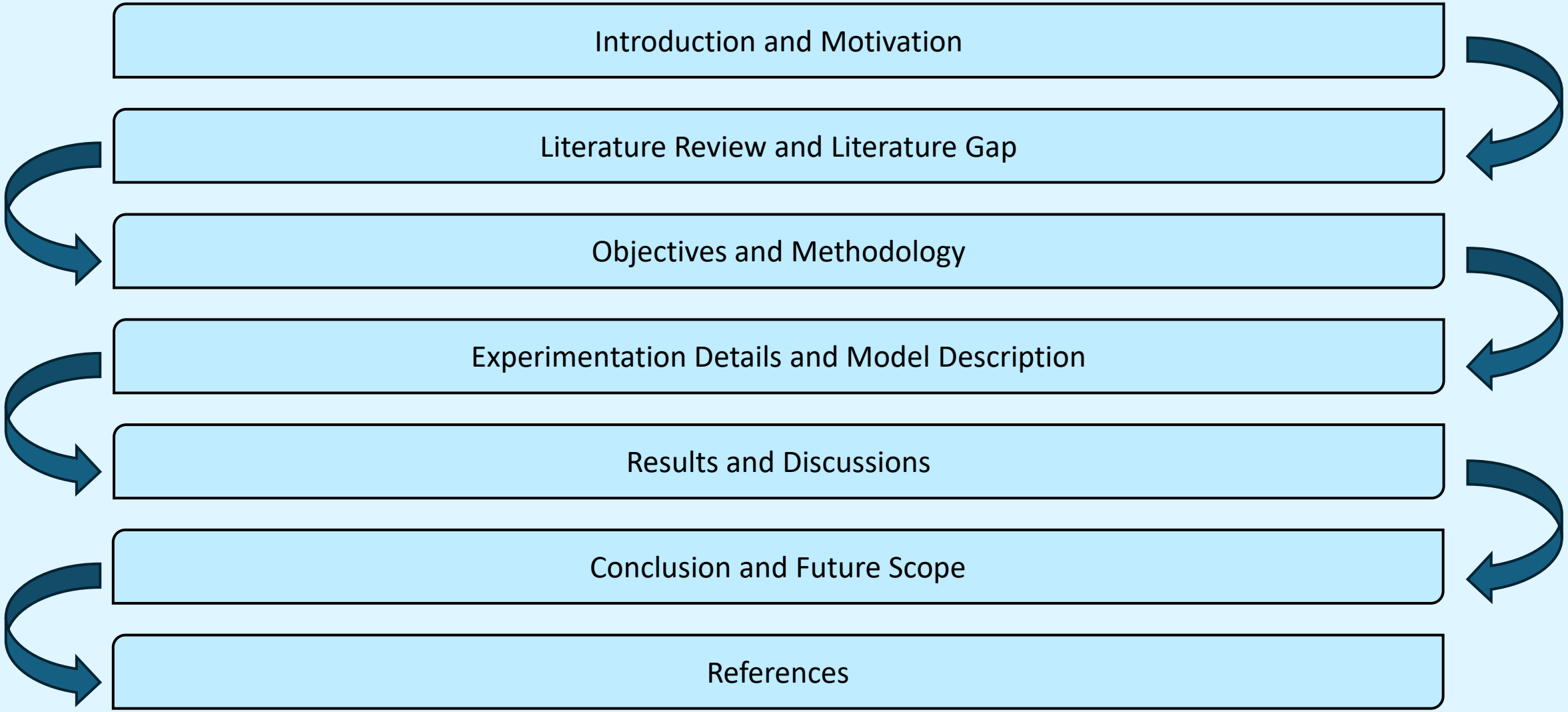
Obstacle Detection in USVs

Kovid Sharma
21ME01026

Under the Supervision of Dr. Yogesh Bhumkar and Dr. Debi Prosad Dogra

School of Mechanical Sciences
Indian Institute of Technology, Bhubaneswar

Content / Outlines



Introduction and Motivation

- Unmanned Surface Vessels(USVs) are generally used in perimeter monitoring and surveillance tasks.
- USVs can be navigated either through manual control or autonomously following a trajectory.
- They come in various dimensions but usually less than 2 meters.
- They are portable and can be easily navigated in narrow marinas.
- In the majority of applications, they are required to navigate along a pre-determined path.
- May be required to perform some tasks autonomously.

Fig. 1

Image credits: https://www.defenceprocurementinternational.com/features/maritime/are-unmanned-surface-vehicles-a-paradigm-shift-in-naval-warfare

Introduction and Motivation

- An autonomous system necessitates the use of an obstacle detection system and requires excessive computation.
- Computation can be performed either at the ground station or via an onboard computing device.
- Performing computation on the ground station and information transfer using telemetry limits the range of the USV.
- For autonomous navigation, variety of sensors are used such as LiDAR, Sonar, Radar, Camera etc.
- The compact size of the USVs constraints the power consumption and payload capacity which limits the use of sensors.

Fig. 2

Image credits: https://defense.info/defense-systems/unmanned-surface-vehicles-in-support-of-the-sustainment-mission/

Introduction and Motivation

- USV needs situational awareness to identify obstructions.
- General obstructions in USVs scenarios are other vessels, people (Scuba divers), objects such as trash and most importantly the shoreline or water-edge line.
- In certain instances (particularly those beyond India), satellite provides seashore maps[3]
- When combined with a GPS, enable the management of seashore obstacles.
- No publicly available datasets of USVs for Indian Scenarios

Fig-3a
Image credits: https://danandholly.com/2021/01/indian-fishing-boats/

Fig-3b
Image taken from MODS dataset[21]

Literature Review and Literature Gap

- Obstacle detection was previously addressed in the field of Unmanned Ground Vehicles (UGVs).
- The research for it in USVs is still in the early stages.
- Methods designed for UGVs cannot be easily applied to USVs for the following reasons:
    - They depend on estimating the ground plane. Not applicable to the underwater environment of USVs.
    - Maritime environment exhibit greater diversity as even a little submerged object pose a risk.
- A lot of methods for detecting obstacles in marine environments is the use of range sensors like Radar, Sonar or LiDAR. Size and power restrictions.
- Therefore, cameras combined with computer vision algorithms may be a promising alternative.

Literature Review and Literature Gap

Authors	Contribution
Larson et al.[4] (2007)	• Used a monocular camera and Estimated horizon using trigonometric calculations and nautical charts.
Gal, O.[5] (2011)	• Used edge detection approach to detect the horizon line. • Assumed sea-edge as a straight line.
Wang et al.[6] (2011)	• Used pixel profile analysis and RANSAC regressor to estimate sea-sky line. • Created a real-time obstacle detection system for a monocular and stereo camera system based on saliency detection

- Most of these approaches approximate the edge of water by the horizon line.
- In coastal scenarios, water-edge doesn't align with the horizon.

Literature Review and Literature Gap

Authors	Contribution
Kristan et al.[7] (2015)	• Introduced a graphical model for monocular obstacle detection by constrained semantic segmentation • Partitions an image into three distinct and approximately parallel regions: sky, ground and water. • Does not presume a straight water edge and operates in real-time • Constructed the first large annotated dataset - MODD
Bovcon et al.[8, 9] (2018, 2019)	• Included sensor data like roll and pitch value from the Inertial Measurement Unit(IMU). • Improved the segmentation model by introducing stereo verification. • Constructed a new dataset with time-synchronizatized data streams – MODD2. • Proposed another dataset MaSTr1325 which was per-pixel semantically labelled and a data augmentation protocol
Bovcon et al.[10] (2022)	• Proposed a large unified dataset – MODS for obstacle detection and estimate depth maps.

Literature Review and Literature Gap

Authors	Contribution
Ahmed et al.[11] (2023)	• Used a generative adversarial network(GAN) model for image de-hazing and de-noising. • YOLOv5 based object detection system to detect objects from enhanced images.

- Majority of approaches used are segmentation based and cannot give real time inference on Embedded GPUs.
- Accuracy of the existing approaches will significantly vary when implemented in Indian Scenarios.
- Deep learning-based object detectors have a significant potential for real-time obstacle detection.
- There is no publicly available dataset for USVs that cater to Indian scenarios.

Objectives and Methodology

With this research, we plan to complete the following objectives:

1. To explore and experiment existing obstacle detection approaches in Indian scenarios.
2. To create an annotated dataset for Indian waters to boost the research in this field.
3. To develop a robust obstacle detection system that can work on an Embedded GPU.

Objectives and Methodology

Methodology to be followed:

- Experimentation with a multi-modal approach for obstacle detection. Semantic segmentation for detecting the water-edge and an object detection model to detect obstacles.
- Exploring multiple segmentation and object detection models to test their real-time performance.
- Since the USV in under development, we plan to experiment on the MODS[10] dataset.
- Developing an IMU – camera calibration system and employing an Embedded GPU. Testing the USV on various water-bodies.
- Construct a dataset with visual and sensor data, and experiment with the models created.

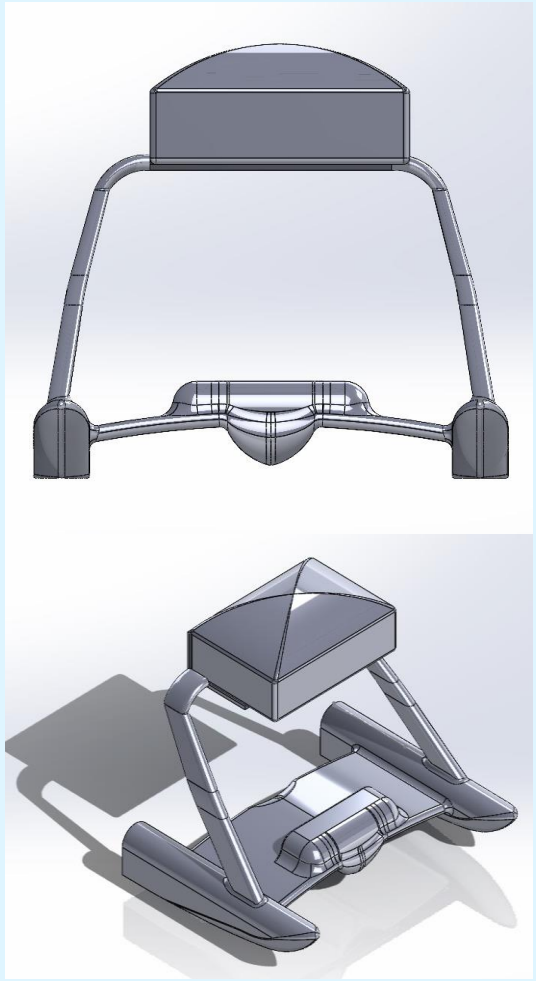


Fig. 4: CAD Model

Experimentation Details and Model Description

Dataset Modification

Segmentation

- The MODS dataset contained annotations in JSON format.
- The dataset was modified for water-edge segmentation task and contains 8175 images.
- Binary Masks of the images were created using OpenCV library.
- The images were resized to 640 x 640 pixels for training.

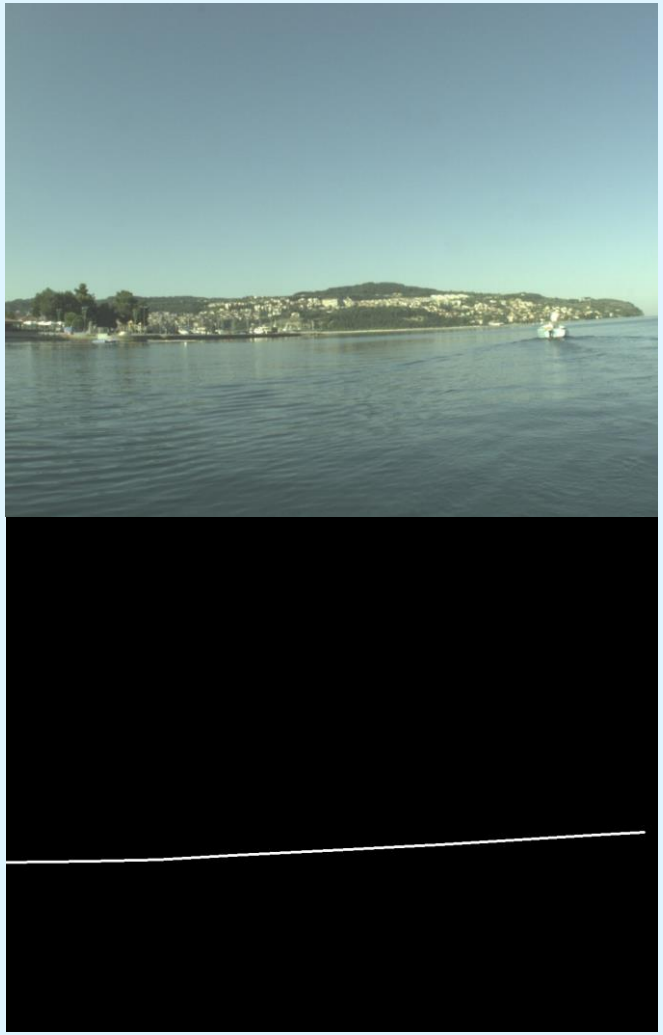


Fig. 5: Image and its mask

Experimentation Details and Model Description

Dataset Modification
Detection

- Bounding Boxes format was changed to midpoint format.
- [topleftx, toplefty, bottomrightx, bottomrighty]
  to
  [midpointx, midpointy, width, height]
- The coordinates were re-scaled between 0 and 1 using image dimension.
- The bounding boxes were then converted to TXT format

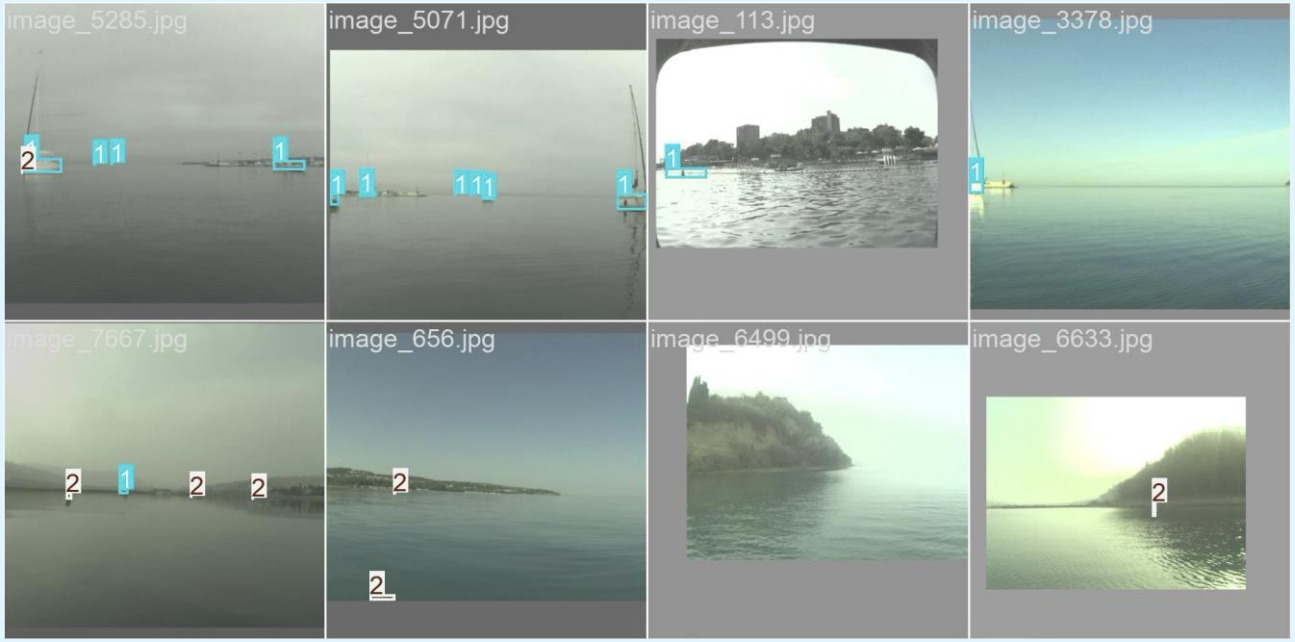


Fig. 6: Examples with bounding boxes

Experimentation Details and Model Description

Model Description

Vanilla U-Net

- The model used for the segmentation task is a modified U-Net Architecture[12].
- An input size of 640 is used instead of 572.
- Batch Normalization layers are included to prevent overfitting.
- Model was created using the PyTorch library and Python Programming language.
- The final model contains 118,577,922 trainable parameters.

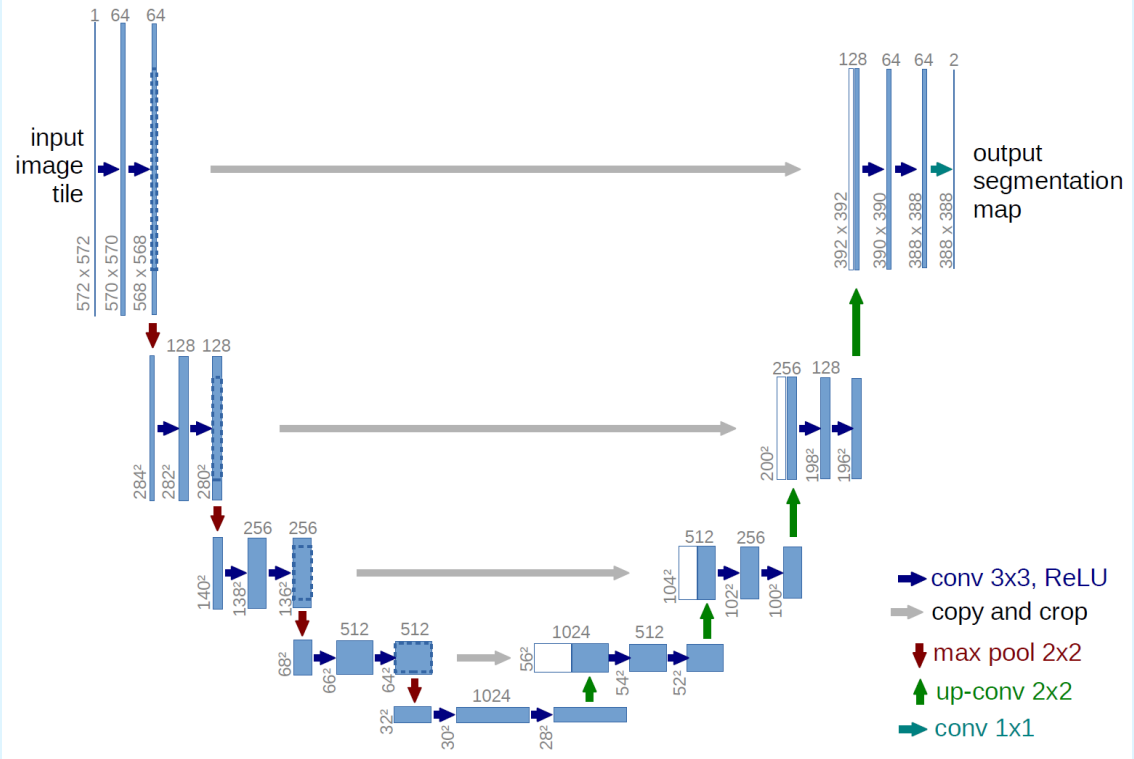


Fig. 7: U-Net Architecture[12]

Experimentation Details and Model Description

Model Description

Models with custom encoders

- All the models were created using Python, Pytorch framework, and Segmentation-Models-Pytorch library.
- List of models and their specifications are mentioned in Table-1

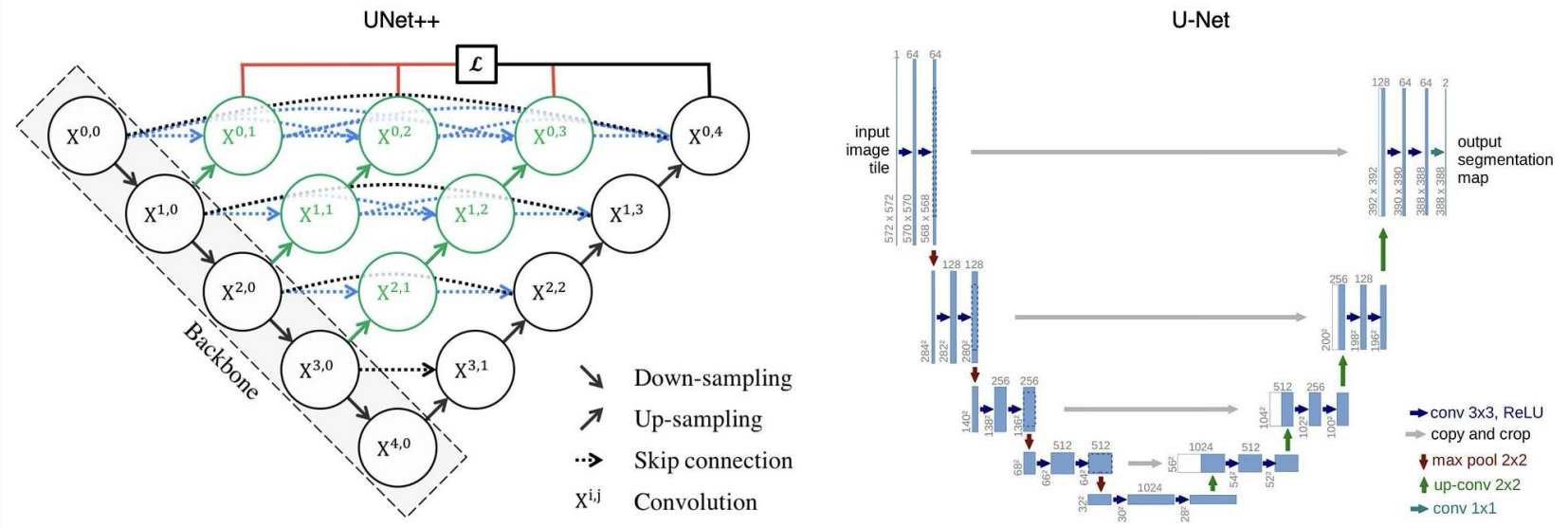


Fig. 8: U-Net++ Architecture

Experimentation Details and Model Description

Table-1: Model Specificaions(Custom Encoder)			
Encoder	**Decoder**	**Model Size(Million)**	**Model Tag**
ResNet18	U-Net	19.834	res-unet
ResNet18	U-Net++	21.753	res-unet++
ResNet18	PSPNet	11.329	pspnet-resnet18
ResNet18	LinkNet	11.663	linknet-resnet18
ResNet18	MANet	28.001	manet-resnet18
MobileNetv2	U-Net	13.388	mobnet-unet
EfficientNet-b2	U-Net	14.746	eff-unet
EfficientNet-b4	PSPNet	17.614	pspnet-effnet-b4
EfficientNet-b4	LinkNet	17.862	linknet-effnet-b4
EfficientNet-b4	MANet	31.276	manet-effnet-b4

Experimentation Details and Model Description

Model Description

Object Detection

- YOLOv8 and RT-DETR are trained for the object detection task
- Both models are State-of-the-art and perform well in real time

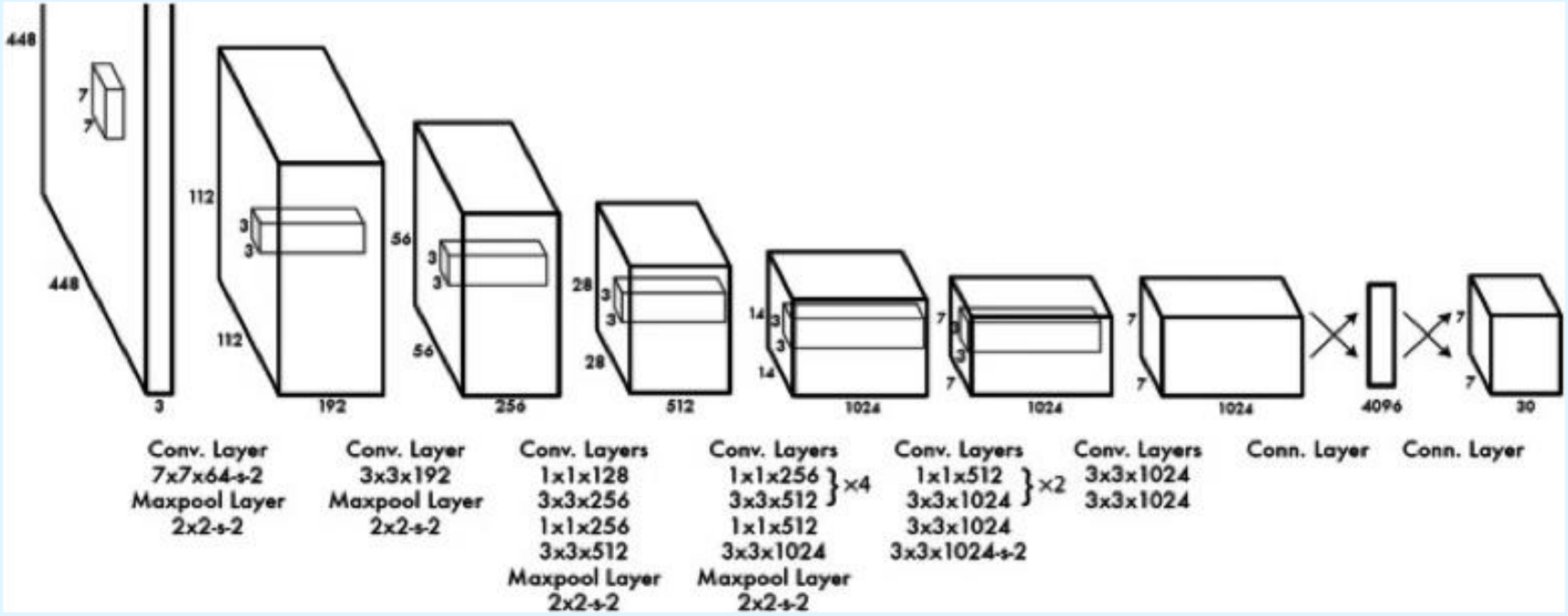


Fig. 9: YOLOv1 Model Overview

Experimentation Details and Model Description

Experimentation Details

Vanilla U-Net

- Model was trained for 4 hours and in total 25 Epochs.
- Two NVIDIA T4 GPUs provided by kaggle are used parallelly.
- The loss-function used in Binary cross entropy with logits loss.
- Optimizer used is the Adam Optimizer.
- The learning rate schedule can be seen in table-2
- Data Augmentations were applied using the albumentations library.

Epoch No.	Learning Rate
1-10	0.001
11-15	0.0001
16-20	0.00001
21-25	0.000001

Table-2: Learning Rate Schedule for vanilla u-net

Experimentation Details and Model Description

Experimentation Details

Segmentation models with custom encoders

- Training Setup - NVIDIA A10 GPU provided by the AI, Data Analytics and IoT Lab, SECS, IIT Bhubaneswar.
- The loss-function used is dice loss. It measures the overlap between predicted and ground truth.
- Optimizer – Adam Optimizer
- The learning rate schedule can be seen in table-3
- Data Augmentations were applied using the albumentations library.
- One model was also trained for 30 Epochs to check convergence.

Epoch No.	Learning Rate
1-5	0.0001
6-10	0.00001
11-15	0.000001

Table 3: Learning Rate Schedule for custom models

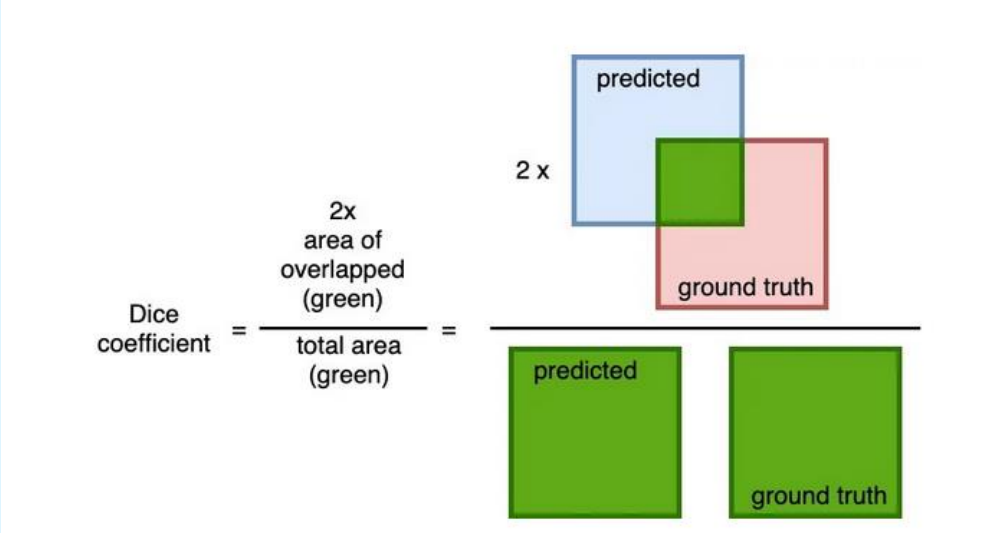


Fig. 10: Dice Coefficient

Image credits: https://cvinvolution.medium.com/dice-loss-in-medical-image-segmentation-d0e476eb486

Experimentation Details and Model Description

Experimentation Details

Object Detection

- Training Setup - NVIDIA A10 GPU provided by the AI, Data Analytics and IoT Lab, SECS, IIT Bhubaneswar.
- IoU Threshold = 0.7, learning rate = 0.01, optimizer = Adam
- The training details are mentioned in Table 4

Table-4: Object Detection Training Details

Model Name	Model Size(Million)	Training Epochs	Batch Size
Yolov8	43.69	100	32
RT-DETR	32.97	50	24

Results and Discussions

- Vanilla U-Net achieved a dice score of 0.68. MANet with EfficientNet-b4 backbone achieved a dice score of 0.66 with 3.8 times fewer parameters
- Dice score > 0.8, considered good.
- Yolov8 achieved 0.698 AP50 and 0.3612 AP95.
- RT-DETR achieved 0.6718 AP50 and 0.3753 AP95
- The AP scores are good but the confusion matrix tells a different story.
- The results and loss curves can be seen in next slides.

Results and Discussions

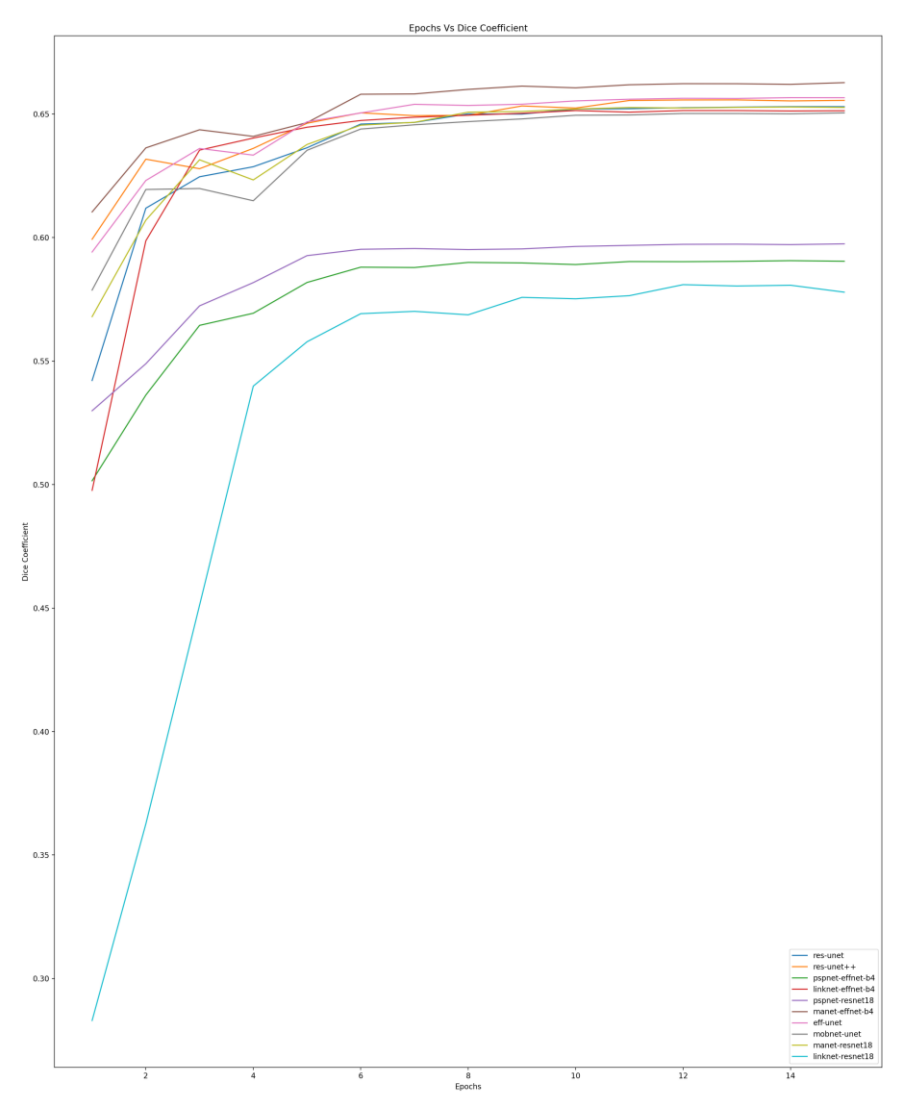


Fig. 11a: Segmentation Dice Curve

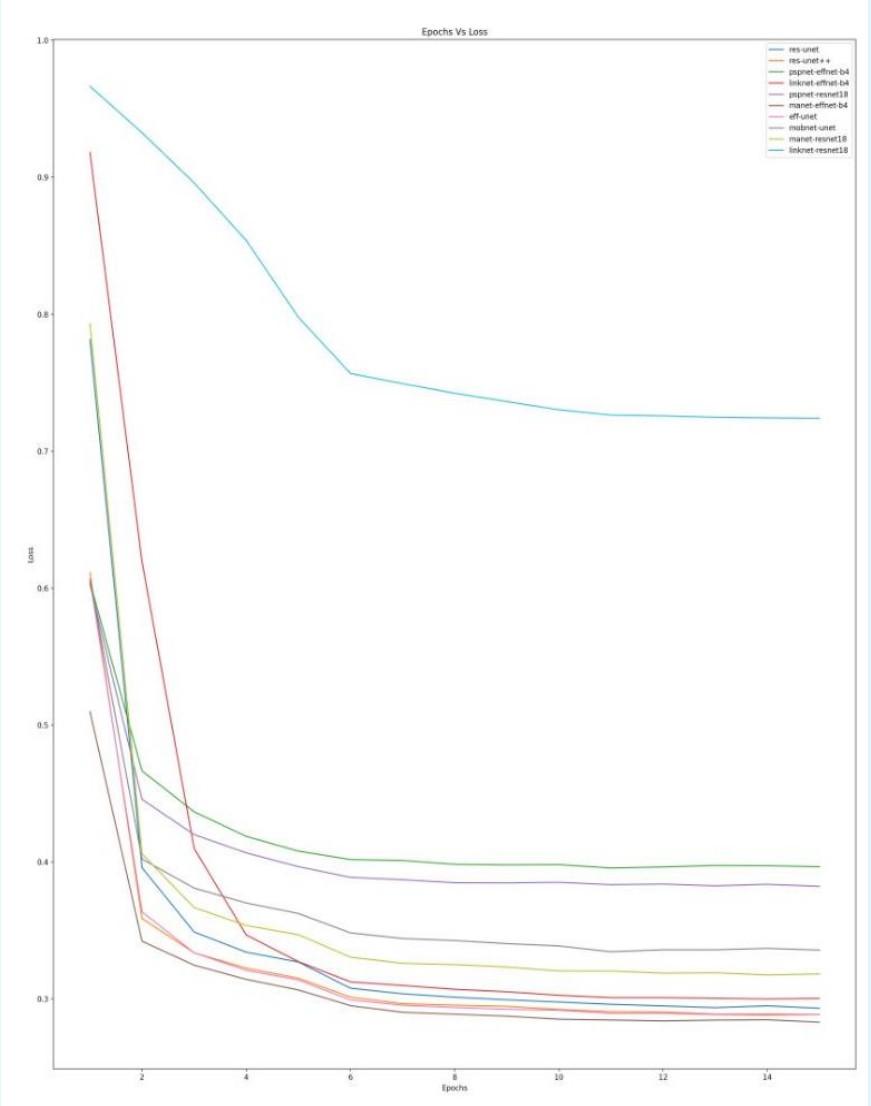
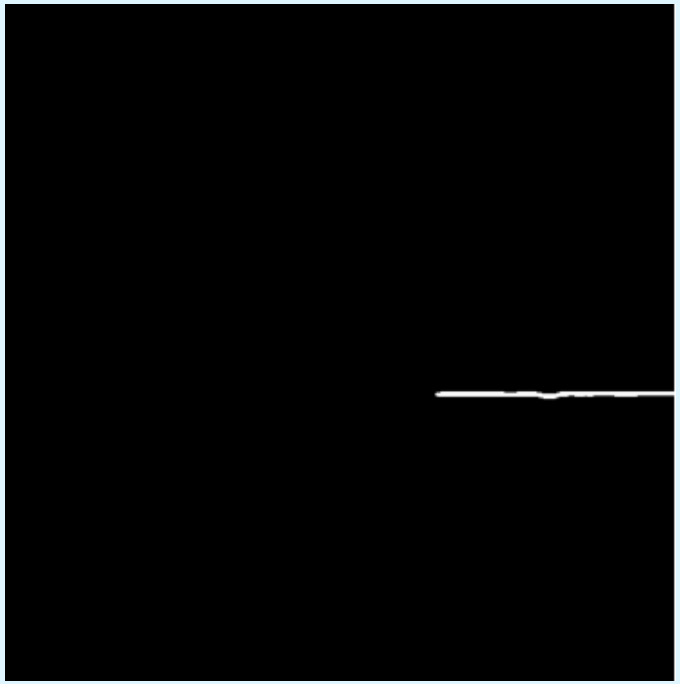


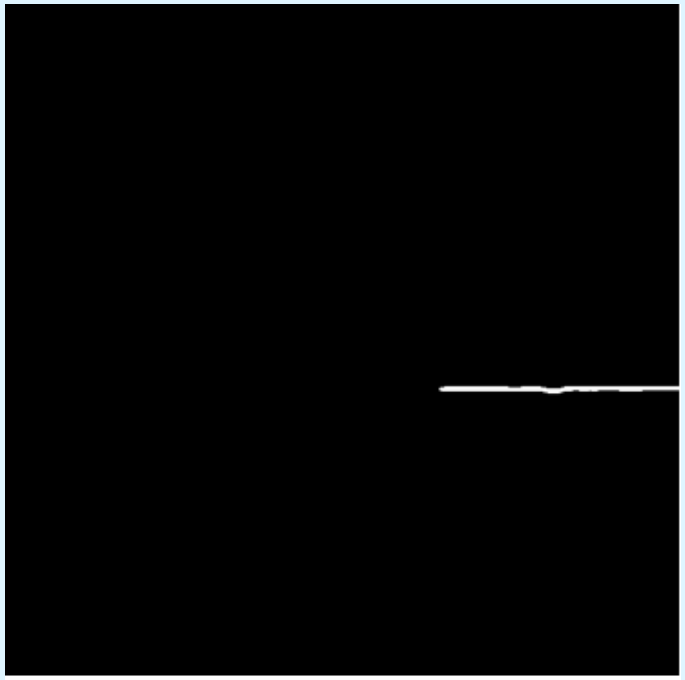
Fig. 11b: Segmentation Loss Curve

Results and Discussion

Input Image



Ground Truth Mask



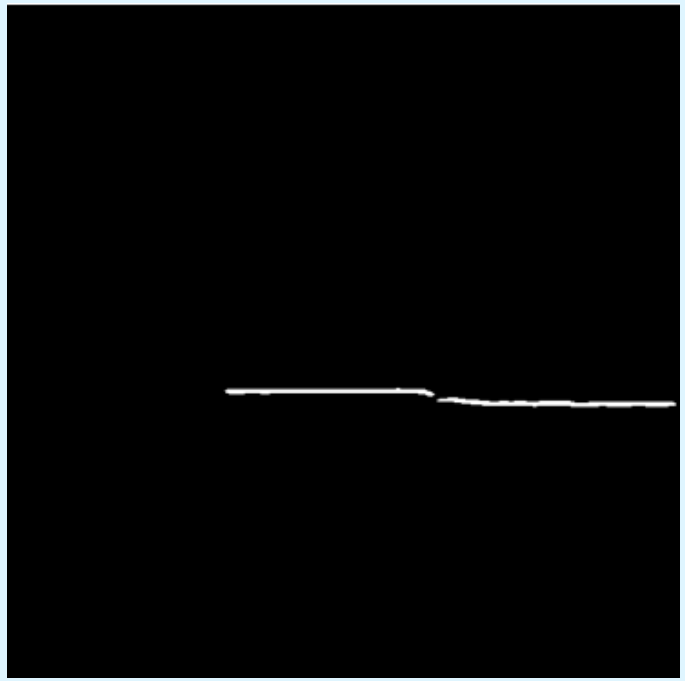
Predicted Mask

Fig. 12a: Segmentation Results

Results and Discussion

Input Image

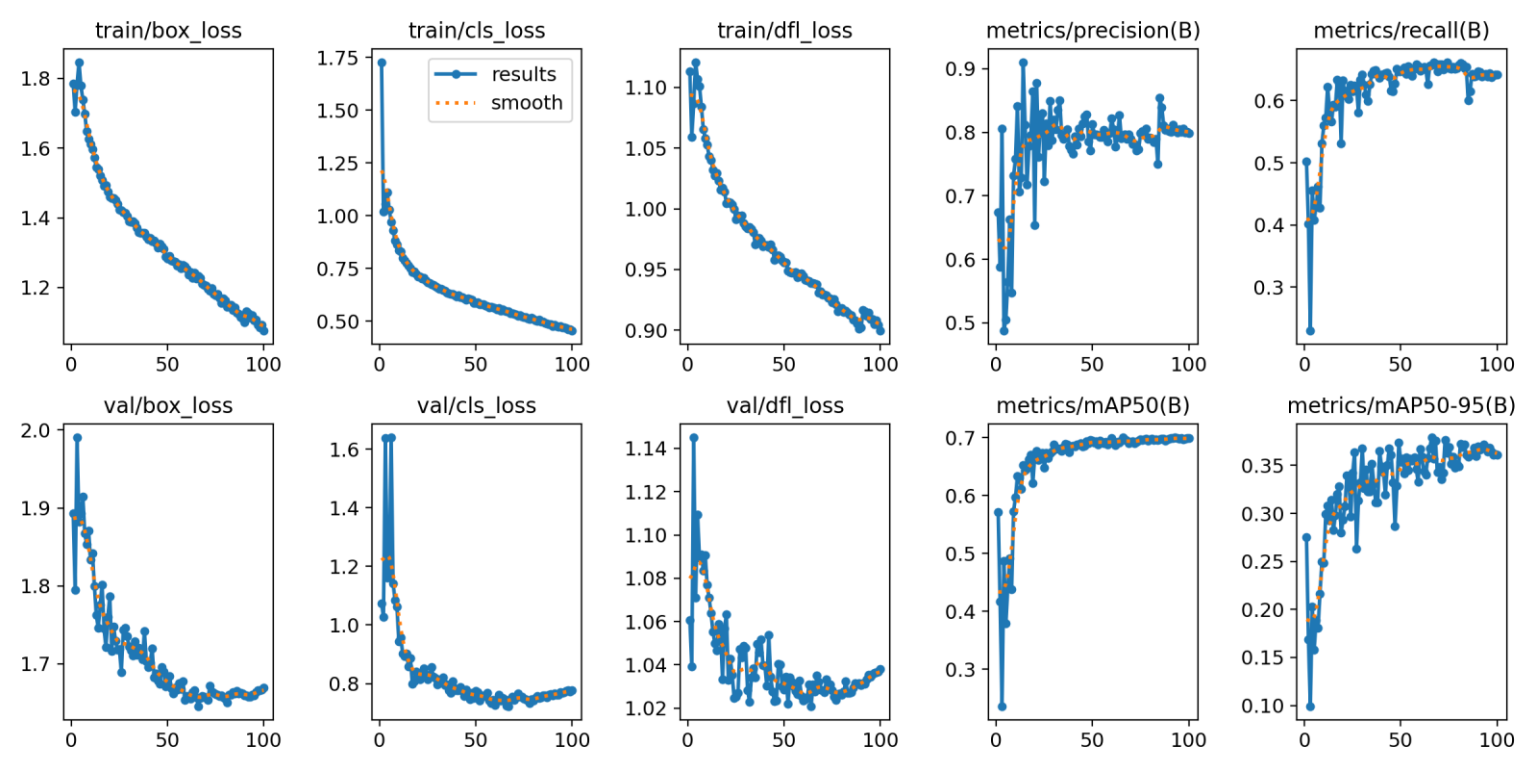
Ground Truth Mask



Predicted Mask

Fig. 12b: Segmentation Results

Results and Discussion



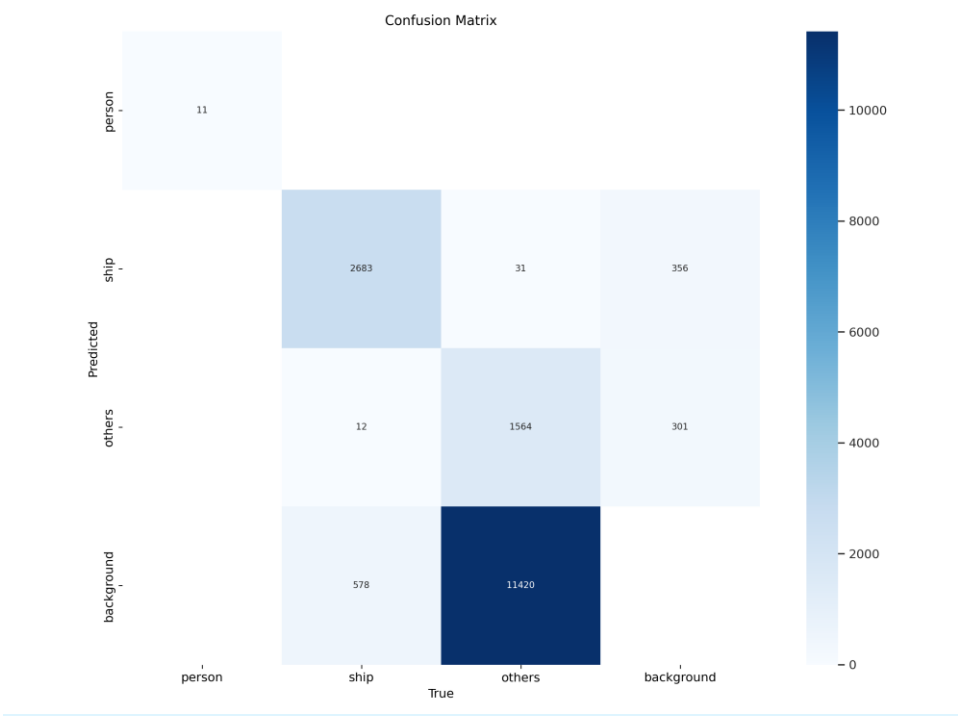
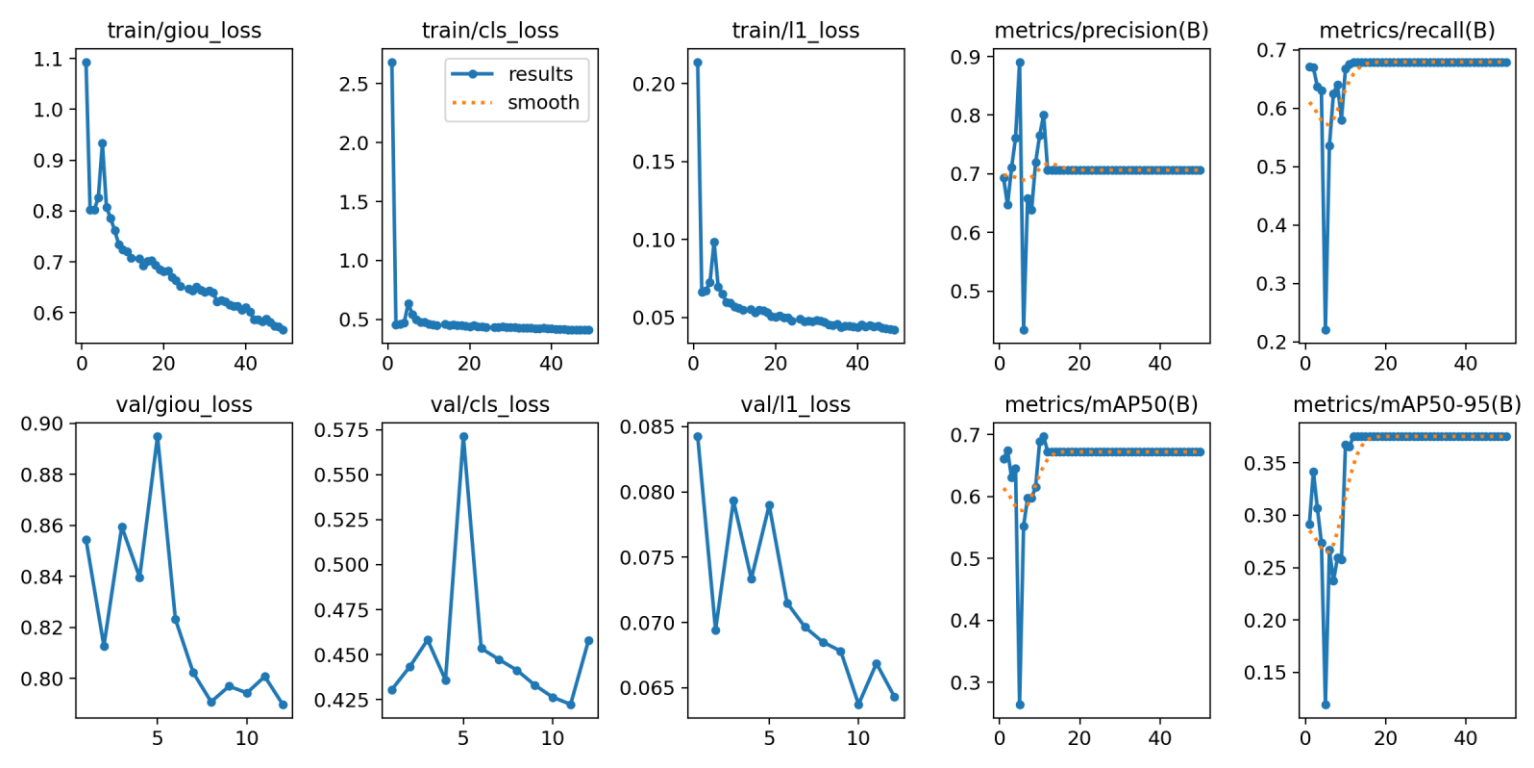


Fig. 13a: YOLOv8 Results

Results and Discussion



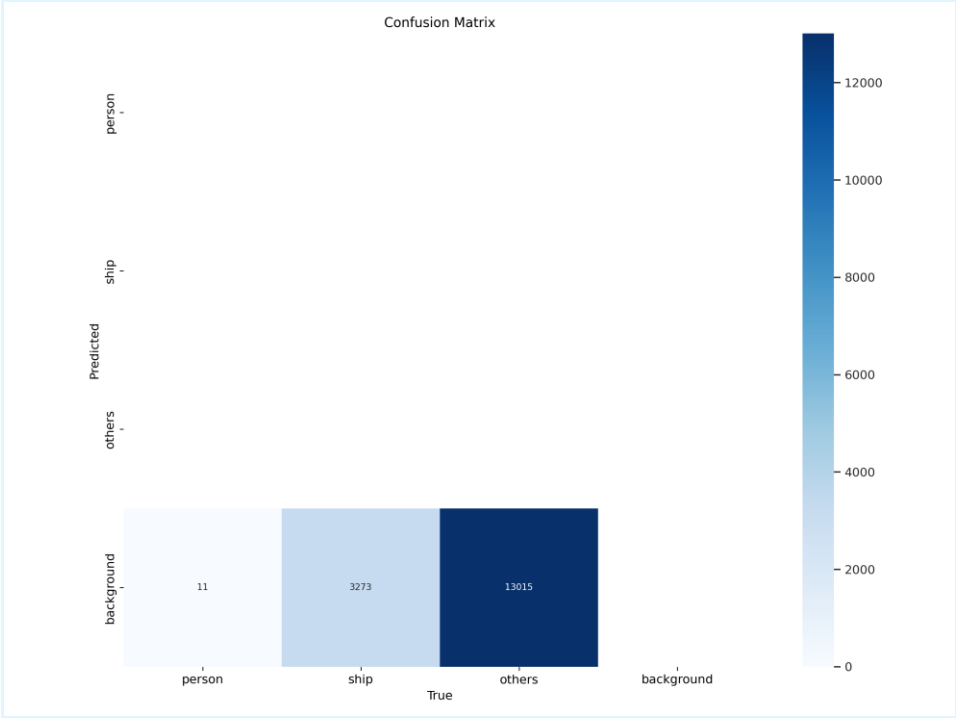


Fig. 13b: RT-DETR Results

Conclusion and Future Scope

- The experimentation done with segmentation and detection models was successful.
- MA-Net with EfficientNet backbone performed the best in semantic segmentation given that the model size was only 31M parameters.
- The data from the IMU can be used to create horizon masks, which can then be used as a feature.
- Sophisticated loss-functions can be used to increase water-obstacle separation.
- YOLOv8 performed best in the detection task. However, it requires a lot of hyper-parameter tuning.
- For further experimentation, models like SSD, FCOS, and YOLOX can be trained.

Conclusion and Future Scope

- To check the real-time capabilities, models need to perform prediction on the embedded GPU.
- depends heavily on the quantisation as well as the run-time used. For example – Nvidia DeepStream
- Experimentation can also be performed for a unified model as we can use all the computation power of the GPUs.
- Construction of an Indian dataset is extremely necessary

References

1. Title slide image Credits: https://www.oglemodels.com/case-studies/unmanned-surface-vessel/
2. Title slide image Credits: https://www.unmannedsystemstechnology.com/expo/autonomous-surface-vehicles-asv/
3. Heidarsson et al. (2011). Obstacle detection from overhead imagery using self-supervised learning for Autonomous Surface Vehicles. IEEE International Conference on Intelligent Robots and Systems. 3160-3165. 10.1109/IROS.2011.6048233.
4. Larson et al. (2007). Advances in Autonomous Obstacle Avoidance for Unmanned Surface Vehicles.
5. Gal, O. (2011). Automatic Obstacle Detection for USV's Navigation Using Vision Sensors. In A.Schlaefer & O. Blaurock (Eds.), Robotic Sailing (pp. 127–140). Springer Berlin Heidelberg.
6. Wang et al. (2011). Real-time obstacle detection for Unmanned Surface Vehicle. 2011 Defense Science Research Conference and Expo (DSR)
7. Kristan et al. (2015). Fast image-based obstacle detection from unmanned surface vehicles.
8. Bovcon et al. (2018). Stereo obstacle detection for unmanned surface vehicles by IMU-assisted semantic segmentation. Robotics and Autonomous Systems, 104, 1–13.
9. Bovcon et al (2019). The MaSTr1325 dataset for training deep USV obstacle detection models. 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 3431–3438.
10. Bovcon et al. (2022). MODS—A USV-Oriented Object Detection and Obstacle Segmentation Benchmark. IEEE Transactions on Intelligent Transportation Systems, 23(8),13403–13418.
11. Ahmed et al. (2023). Vision-Based Autonomous Navigation for Unmanned Surface Vessel in Extreme Marine Conditions.
12. Olaf Ronneberger, Philipp Fischer, Thomas Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation." (2015).

THANK YOU